Model Documentation

TalentMatch AI — Matching & Ranking Methodology

Task ID: AI-2 | TEYZIX CORE Internship — June Batch 2026

1. Overview

This document explains the methodology, algorithms, and scoring logic used by TalentMatch AI to evaluate and rank candidates. No external ML model or LLM API call is required to run the core matching engine — all scoring is deterministic, rule-based, and computed directly from text, which makes the system fully explainable and reproducible without relying on a paid API or internet access at inference time. (An optional LLM-based mode is available separately in the AI-1 summarizer companion tool, but is not part of this matching engine.)

2. Information Extraction Model

Resume parsing uses rule-based pattern matching (regular expressions) rather than a trained NER model. This was a deliberate choice: the task evaluation criteria weight explainability and system design highly (30% combined), and regex-based extraction is 100% traceable — every extracted field can be traced back to the exact pattern that matched it, with zero hallucination risk.

Field	Extraction Method
Email	Regex: standard email pattern (local@domain.tld)
Phone	Regex: international/local phone number patterns with optional country code
Name	Heuristic: first short, capitalized, non-email line within the first 6 lines of the document
Skills	Dictionary lookup against a curated 90+ term skill database using word-boundary regex matching
Experience (years)	Regex: captures patterns like '4 years', '3+ years of experience'
Education	Keyword filter: lines containing degree/institution keywords (bachelor, university, etc.)
Summary	Extractive: first 2 sentences of the document longer than 4 words, truncated to 220 characters

3. Skill Matching Model

A fixed skill dictionary (SKILL_DB) of 90+ common technical and soft skills is matched against both the job description and each resume using case-insensitive word-boundary regex matching. This avoids false positives (e.g. matching 'r' inside 'experience') while remaining fast and dependency-free.

```
skill_score = |matched_skills| / |jd_required_skills|
```

Where matched_skills is the intersection between a candidate's extracted skills and the job description's required skills. If the job description has no skills detected, this score defaults to 0.5 (neutral) so that semantic similarity carries more weight instead.

4. Semantic Similarity Model

Beyond exact skill-keyword matching, the system computes a semantic similarity score using a Term-Frequency vector space model and cosine similarity — the same foundational approach behind classical TF-IDF information retrieval, simplified to avoid requiring an external corpus for IDF weighting (since each run only has one job description and a handful of resumes, corpus-wide IDF statistics would not be meaningful).

Steps:

1. Tokenize both the job description and resume text into lowercase words, removing stopwords and tokens shorter than 3 characters
2. Build the shared vocabulary: the union of all unique tokens from both documents
3. Construct a term-frequency vector for each document over that shared vocabulary
4. Compute cosine similarity between the two vectors:

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

This produces a 0–1 score representing how textually similar the overall resume is to the job description, capturing relevant context that isn't covered by the fixed skill dictionary (for example, domain terminology, project descriptions, and responsibilities phrased differently than the JD).

5. Experience Scoring Model

```
experience_score = min(candidate_years / min_required_years, 1.3) / 1.3
```

This rewards candidates who meet or exceed the minimum required experience, with a soft cap so that drastically over-qualified candidates (e.g. 15 years for a role requiring 3) don't disproportionately dominate the score over actually relevant skill/semantic fit. If no minimum is specified, candidates with any detected experience receive 0.8, and those with none receive 0.5 (neutral, since experience may simply not have been stated in years).

6. Final Weighted Score & Ranking Priority Modes

The three component scores (skill, semantic, experience) are combined using a weighted sum, where the weights are selectable via the --priority flag (CLI) or the Ranking Priority dropdown (web UI):

Priority Mode	Skill Weight	Semantic Weight	Experience Weight
balanced (default)	0.45	0.30	0.25
skills	0.60	0.20	0.20
experience	0.30	0.20	0.50
semantic	0.20	0.60	0.20

```
final_score = skill_score × w_skill + semantic_score × w_semantic + experience_score × w_experience
```

7. Why This Approach Was Chosen

- **Explainability (10% of evaluation criteria):** every score breaks down into 3 transparent components, plus the exact lists of matched and missing skills — there is no opaque neural network output to justify after the fact.
- **Zero external dependencies at inference time:** the core engine needs no API key, no GPU, and no internet connection, making it trivially deployable and fully reproducible.
- **Determinism:** the same resume and job description will always produce the exact same score, which matters for fairness and auditability in a hiring context.
- **Tunable for different hiring priorities:** the 4 priority modes let a recruiter weight skills, experience, or contextual fit differently depending on the role being filled.

8. Known Limitations

- The skill dictionary is fixed and curated — a skill not in SKILL_DB (e.g. a niche or very new technology) will not be detected by keyword matching, though it can still contribute to the semantic similarity score.
- Name extraction is heuristic and can occasionally misfire on resumes with unusual header formatting (e.g. a name split across two lines, or a resume that leads with a section title instead of the candidate's name).
- PDF text extraction quality depends on how the PDF was generated — scanned image-based PDFs with no embedded text layer cannot be parsed without a separate OCR step, which is intentionally out of scope to keep the system lightweight.
- Semantic similarity here is a simplified TF-cosine model, not a transformer-based embedding model (e.g. Sentence-Transformers) — it captures lexical overlap well but will miss true paraphrase-level similarity (e.g. 'led a team' vs 'managed engineers'). Swapping in a Sentence-Transformers embedding model is a natural upgrade path noted in Section 9.

9. Possible Future Upgrades

- Replace the TF-cosine semantic model with Sentence-Transformers embeddings + a vector database (FAISS/ChromaDB/Pinecone) for true semantic (not just lexical) similarity
- Use an LLM (Claude/GPT) to generate natural-language candidate summaries and interview questions tailored to each candidate's gaps — listed as a bonus feature in the task brief
- Train a lightweight classifier on labeled hire/no-hire outcomes to learn optimal weights instead of the fixed weight presets used currently